Ruta Fantasma

Memoria del proyecto Scrum — Desarrollo de Interfaces

Product Owner: Daniel Martín Scrum Master: Aitor
Desarrollo: Asier García Catalán, Daniel Rodríguez, Adam

1º DAM — Curso 2026-2027

Índice

1 Introducción **1**

2 El producto **1**
2.1 Visión 1
2.2 Usuarios 1
2.3 Product Backlog inicial 1

3 Equipo y forma de trabajo **2**
3.1 Roles 2
3.2 Tecnologías (stack) 2
3.3 Herramientas 2
3.4 Ceremonias 3

4 Sprint 1 **3**
4.1 Sprint Planning 3
4.2 Trabajo realizado 3
4.3 Configuración del repositorio y del flujo de trabajo 3
4.4 DT2 · Preparación y publicación: proyecto Angular con zonas y PWA . . 4
4.5 Organización del árbol del proyecto 5
4.6 Carpeta de diseño para los mockups 6
4.7 PS-21 · Servicio HTTP base de Angular 6

5 Conclusiones **7**

1 Introducción

Esta memoria recoge el desarrollo del proyecto **Ruta Fantasma**, realizado en la asignatura de Desarrollo de Interfaces siguiendo la metodología Scrum.

(Completad aquí: objetivo del trabajo, motivación de la idea y qué se espera conseguir al final.)

2 El producto

2.1 Visión

Ruta Fantasma es una aplicación de reloj para correr contra tu mejor marca. En pantalla aparecen dos puntos: tú y tu “sombra” (tu mejor carrera en ese recorrido). Durante la carrera, la app avisa por sectores en verde (vas mejor) o en rojo (vas peor), como los tiempos por sector de la Fórmula 1.

2.2 Usuarios

(Quién va a usar la app y qué necesita.)

2.3 Product Backlog inicial

ID	Historia de usuario	Prioridad	Estimación
HU-01	Como corredor, quiero ver mi posición y la de mi sombra para saber si voy mejor o peor	Alta	*(pts)*
HU-02	Como corredor, quiero recibir avisos verdes/rojos por sector para ajustar el ritmo	Alta	*(pts)*
HU-03	*(…)*		

3 Equipo y forma de trabajo

3.1 Roles

Rol	Persona	Responsabilidad
Product Owner	Daniel Martín	Define y prioriza el Product Backlog
Scrum Master	Aitor	Facilita las ceremonias y elimina impedimentos
Desarrollador	Asier García Catalán	Desarrollo
Desarrollador	Daniel Rodríguez	Desarrollo
Desarrollador	Adam	Desarrollo

3.2 Tecnologías (stack)

Parte	Tecnología	Motivo
Frontend	Angular + TypeScript	Vista en clase; permite simular la pantalla del reloj en el navegador
Backend	Node.js + Express	JavaScript en servidor, conocido por el equipo
Base de datos	MongoDB	Guarda de forma natural las rutas (listas de puntos GPS y tiempos)

3.3 Herramientas

- **GitHub:** repositorio común del código y de esta memoria.
- **GitHub Issues y Projects:** Product Backlog y tablero del sprint (Por hacer / En curso / Hecho).
- **Claude Code:** asistente de programación que usa cada desarrollador. Todos comparten el mismo contexto mediante el archivo `CLAUDE.md` del repositorio.
- **Generación automática de la memoria:** cada avance se documenta en Markdown y GitHub Actions genera este PDF en cada subida.

3.4 Ceremonias

- **Sprint Planning:** al inicio de cada sprint.
- **Daily:** *(frecuencia real del equipo)*.
- **Sprint Review y Retrospectiva:** al final de cada sprint; se resumen en el apartado de cada sprint.

4 Sprint 1

Fechas: *(inicio — fin)*

Objetivo del sprint: *(una frase)*

4.1 Sprint Planning

Historia / tarea	Responsable	Estimación
(HU-01 …)	*(nombre)*	*(pts)*

4.2 Trabajo realizado

4.3 Configuración del repositorio y del flujo de trabajo

Autor: Asier García · **Fecha:** 13/10/2026 · **Issue:** —

4.3.1 Qué se ha hecho

Se ha preparado el repositorio común para que los tres desarrolladores trabajen con el mismo contexto y para que la memoria se genere sola.

4.3.2 Cómo se ha hecho

- Se creó el archivo `CLAUDE.md` con el contexto del proyecto y las reglas del equipo. El asistente de cada desarrollador lo lee al empezar, de modo que los tres trabajan con la misma información.
- Se añadieron dos comandos, `/empezar` y `/terminar`, que automatizan actualizar el repositorio al empezar y documentar, hacer commit y subir el trabajo al terminar.
- La memoria se escribe en Markdown dentro de `docs/memoria/`. Cada tarea terminada genera un archivo propio, lo que evita conflictos entre compañeros.
- Un flujo de GitHub Actions convierte todos esos archivos en `docs/memoria.pdf` cada vez que se sube un cambio a la rama principal.

4.3.3 Por qué

Reduce el tiempo dedicado a tareas repetitivas (subir cambios, maquetar la memoria) y garantiza que la documentación esté siempre al día con el código.

4.3.4 Problemas encontrados

Ninguno por ahora.

4.4 DT2 · Preparación y publicación: proyecto Angular con zonas y PWA

Autor: Asier García · **Fecha:** 14/10/2026 · **Issue:** DT2

4.4.1 Qué se ha hecho

Se ha creado la base del frontend de Ruta Fantasma: un proyecto Angular dividido en tres zonas independientes (`/reloj`, `/app` y `/web`) y configurado como aplicación web progresiva (PWA), de modo que se puede instalar en el móvil y funciona sin conexión. Cada zona tiene ya una primera pantalla de ejemplo sobre la que el resto del equipo puede seguir trabajando.

4.4.2 Cómo se ha hecho

- **Creación del proyecto.** Con Angular CLI 22 se generó el proyecto en `src/frontend`, con enrutado, CSS y sin renderizado en servidor (`ng new ruta-fantasma --routing --style=css --ssr=false`).
- **Zonas con carga diferida.** En `src/app/app.routes.ts` se definieron las tres zonas mediante `loadChildren`. Cada una vive en su propia carpeta `src/app/zonas/<zona>/` con su archivo de rutas (`reloj.routes.ts`, `app.routes.ts`, `web.routes.ts`) y un componente de inicio. La raíz `/` y cualquier dirección desconocida redirigen a `/web`.
  - **`/web`:** página pública de presentación del producto, con titular y tres tarjetas de características.
  - **`/app`:** diseño de móvil con la lista "Mis rutas" (vacía de momento) y un menú inferior.
  - **`/reloj`:** esfera redonda que simula la pantalla del reloj, con los dos puntos (corredor y sombra), el tiempo y la diferencia del sector. El borde cambia a verde o rojo según se vaya mejor o peor. Usa *signals* de Angular y datos de ejemplo hasta que se conecte el GPS.
- **Estilos comunes.** En `src/styles.css` se definieron variables CSS (colores de fondo, morado "fantasma", verde y rojo de sector) para que las tres zonas tengan una imagen coherente.
- **PWA.** Con `ng add @angular/pwa` se añadieron el *service worker* (`ngsw-config.json`), el manifiesto y los iconos. Después se personalizó el manifiesto (nombre "Ruta Fantasma", idioma español, colores del tema) y el `index.html` (idioma, título, descripción, `theme-color` e icono para iOS).
- **Pruebas.**
  - **Pruebas unitarias:** se escribieron con Vitest para comprobar que cada ruta carga su zona, que las redirecciones funcionan y que el reloj formatea bien el tiempo (`12:34`) y la diferencia (`-2,4 s`). Pasan las 10.
  - **Pruebas en el navegador:** se comprobaron las tres zonas en tamaño de escritorio y de móvil. Además, sobre la versión de producción se verificó que el *service worker* se instala, guarda la aplicación en caché y que `/reloj` se abre con el servidor apagado.

4.4.3 Por qué

- **Tres zonas separadas:** cada desarrollador puede trabajar en una zona sin tocar los archivos de los demás, lo que reduce los conflictos en Git. Además, con la carga diferida el navegador solo descarga el código de la zona que se

está usando. La compilación genera un fichero distinto por zona, de 1 a 3 kB cada uno.
- **PWA:** Ruta Fantasma se usa corriendo por la calle, donde la conexión puede fallar. Con el *service worker* la aplicación sigue funcionando sin red y puede instalarse como si fuera una app nativa, sin tener que publicarla en una tienda de aplicaciones.
- **Angular 22 con componentes *standalone* y *signals*:** es la forma recomendada actualmente por Angular y simplifica el código.

4.4.4 Problemas encontrados

- Los tests generados por defecto fallaban porque los componentes usan `routerLink` y el test no tenía router. Se resolvió añadiendo `provideRouter` en cada test.
- En las pruebas de rutas se comprobaba mal qué componente aparecía en pantalla. Se cambió a `RouterTestingHarness.navigateByUrl(url, Componente)`, que verifica directamente que se muestra el componente esperado.
- El *service worker* no se activa con `npm start` (modo desarrollo), solo en la versión de producción (`ng build`). Es el comportamiento normal de Angular y queda anotado en `CLAUDE.md`.

4.5 Organización del árbol del proyecto

Autor: Asier García · **Fecha:** 20/10/2026 · **Issue:** —

4.5.1 Qué se ha hecho

Se ha definido la estructura de carpetas completa del repositorio, tanto del frontend como del backend, para que cada parte del código tenga un lugar fijo antes de empezar a desarrollar las historias de usuario.

4.5.2 Cómo se ha hecho

- En el frontend (`src/frontend/src/app/`) se añadieron, junto a las `zonas/` ya existentes, dos carpetas nuevas: `core/` (servicios y modelos de datos que se usan en toda la aplicación) y `compartido/` (componentes y utilidades reutilizables entre zonas).
- Se creó el esqueleto del backend en `src/backend/`, organizado por capas: `api/` (endpoints), `controladores/`, `servicios/` (lógica de negocio), `modelos/` (MongoDB), además de `config/`, `middlewares/`, `utilidades/` y `tests/`.
- Se añadió un `.env.example` con las variables necesarias y un `README.md` del backend explicando cada carpeta.
- Se actualizó el árbol de `CLAUDE.md` para que los asistentes de todo el equipo conozcan la nueva estructura.

4.5.3 Por qué

Con una estructura acordada desde el principio, los desarrolladores saben dónde colocar cada archivo y se reducen los conflictos al trabajar en paralelo. La separación por capas del backend facilita además probar y modificar cada parte por separado.

4.5.4 Problemas encontrados

Ninguno.

4.6 Carpeta de diseño para los mockups

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** —

4.6.1 Qué se ha hecho

Se ha creado en el repositorio la carpeta `docs/diseno/`, destinada a guardar los mockups de las pantallas de Ruta Fantasma antes de programarlas. Incluye una guía que explica cómo exportar y nombrar los diseños, y una tabla para seguir el estado de cada pantalla.

4.6.2 Cómo se ha hecho

- Se creó `docs/diseno/mockups/`, donde se subirán las imágenes PNG exportadas desde Figma.
- Se añadió `docs/diseno/README.md` con el espacio para el enlace al archivo de Figma, los pasos para exportar un mockup, la regla de nombres (`zona-pantalla.png`, con las mismas zonas que el frontend: `reloj`, `app` y `web`) y una tabla con las cinco pantallas previstas para el primer diseño y su estado (Pendiente, En revisión o Validado).
- Se actualizó el árbol del repositorio y la tabla de decisiones de `CLAUDE.md`.

4.6.3 Por qué

Figma guarda los diseños en la nube, por lo que no quedarían registrados en el repositorio. Subir las imágenes exportadas permite que todo el equipo y el profesor consulten los diseños directamente desde GitHub y que quede constancia de cómo evolucionan. Usar los mismos nombres de zona que el frontend facilita relacionar cada mockup con el código de su pantalla, y la columna de estado deja claro qué diseños ha validado ya el Product Owner.

4.6.4 Problemas encontrados

Se descartó el nombre `diseño` para la carpeta porque las tildes en los nombres de carpetas pueden dar problemas en algunos sistemas y herramientas; se usó `diseno`.

4.7 PS-21 · Servicio HTTP base de Angular

Autor: Asier García · **Fecha:** 21/10/2026 · **Issue:** PS-21

4.7.1 Qué se ha hecho

Se ha creado la capa que comunica el frontend con el backend: un servicio HTTP base (`Api`) que usarán todos los demás servicios, un interceptor que traduce los errores del servidor a mensajes en español y la configuración de la dirección de la API por entornos.

4.7.2 Cómo se ha hecho

- **Entornos.** Con `ng generate environments` se crearon `src/environments/environment.t` (producción) y `environment.development.ts` (desarrollo). Ambos guardan `apiUrl`, que de momento apunta al backend Spring Boot local (`http://localhost:8080/api`). Angular sustituye un archivo por otro automáticamente según el tipo de compilación.
- **Servicio `Api`** (`core/servicios/api.ts`). Envuelve el `HttpClient` de Angular con cuatro métodos genéricos (`get`, `post`, `put` y `delete`). Recibe solo la ruta relativa (por ejemplo `'rutas'`) y construye la URL completa. Se usa el nuevo decorador `@Service()` de Angular 22, que registra el servicio automáticamente en la inyección de dependencias.
- **Interceptor de errores** (`core/interceptores/errores-interceptor.ts`). Es un interceptor funcional que se ejecuta en todas las peticiones. Cuando una falla, convierte el `HttpErrorResponse` en un objeto `ErrorApi` (`core/modelos/error-api.ts`) con el código, un mensaje en español y, si los hay, los errores de validación por campo. Está adaptado a las respuestas reales del backend de Adam: `{ error: '...' }` en los errores generales y `{ campo: 'mensaje' }` en las validaciones (400).
- **Registro.** En `app.config.ts` se añadió `provideHttpClient(withFetch(), withInterceptors([erroresInterceptor]))`.
- **Pruebas.** Con `HttpTestingController` se simulan las respuestas del servidor sin necesidad de tener el backend arrancado. Se comprueba que se construyen bien las URL y los parámetros, que cada método usa el verbo HTTP correcto y que los errores 0 (sin conexión), 400, 404 y 500 se traducen correctamente. Se añadieron 6 pruebas y todas pasan.

4.7.3 Por qué

- **Un único punto de acceso al backend:** si cambia la dirección del servidor, se cambia en un solo archivo, y ningún componente necesita conocer `HttpClient`.
- **Errores centralizados:** las pantallas solo tienen que mostrar `error.mensaje`, sin repetir en cada una la lógica para interpretar los códigos HTTP. El usuario recibe siempre mensajes comprensibles en español.
- **`withFetch()`:** hace que Angular use la API `fetch` del navegador, que es la opción recomendada actualmente.

4.7.4 Problemas encontrados

- Angular 22 genera los servicios con el decorador `@Service()` en lugar del clásico `@Injectable({ providedIn: 'root' })`. Se consultó su definición para confirmar que el servicio se registra automáticamente en toda la aplicación.
- El backend no puede arrancarse en este equipo porque no tiene Java ni MongoDB. Por eso las pruebas simulan el servidor con `HttpTestingController`.

5 Conclusiones

(Se completa al final del proyecto: resultado obtenido, qué se aprendió, qué se mejoraría.)